

DeepAgents 实战：多 Agent 架构的深度调研助手

前面学了 DeepAgents 的各种 middleware，但是太散了。

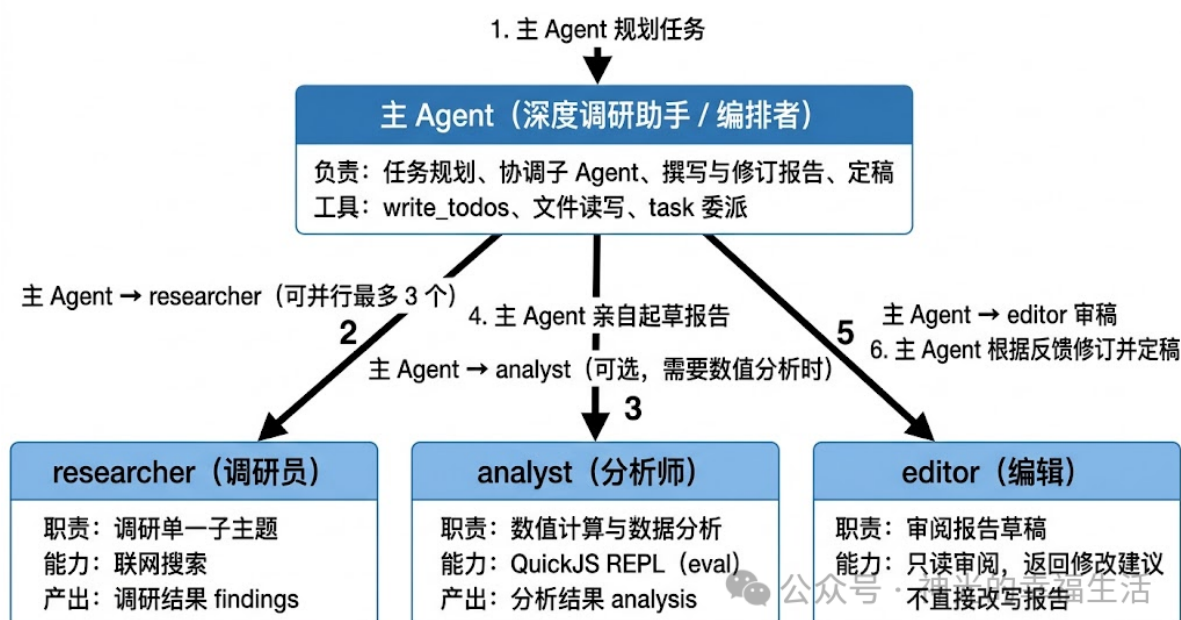
而且 middleware 里带的 tool 需要在 prompt 里说明怎么用。

那有没有一个整合所有中间件的 api，并且内置了 prompt 呢？

有的，就是 createDeepAgent

我们基于 deepagents 开发一个多 Agent 项目：深度调研助手

你只要给它一个主题，它的主 Agent 会自动规划任务，列出 todo 列表，然后交给不同的子 Agent 来执行任务，比如联网搜索、代码执行等，最后生成一份调研报告



我们分了三个子 Agent：

主 Agent：整个系统的编排中心，负责把用户输入的调研主题拆解成可执行流程，并协调各子 Agent 分工完成。它不亲自包揽所有调研细节，而是按「规划 → 调研 → 分析 → 起草 → 审阅 → 定稿」推进任务：先用待办列表明确步骤，再按需委派子 Agent，最后由自己整合材料、撰写报告并根据编辑反馈修订定稿。

调研员子 Agent (researcher)：每次只负责一个聚焦的子主题。通过联网搜索收集资料，将关键事实与来源 URL 整理成结构化摘要，写入 findings_*.md。多个调研员可并行工作，适合把大主题拆成若干子方向同时推进。

分析师子 Agent (analyst)：当调研涉及数字对比、排名、增长率等计算时启用。在 QuickJS REPL 中执行 JavaScript 完成数值分析，禁止凭猜测给出数字。结果写入 analysis_*.md，供主 Agent 写报告时引用。

编辑子 Agent (editor)：在报告草稿完成后介入，从准确性、结构完整性、来源引用、语言表述等维度审阅，返回具体修改建议。编辑不直接改写报告，审阅与修订分离，便于主 Agent 在保持整体思路的前提下做针对性修改。

大概有这个 4 个 Agent

创建项目：

```
1 mkdir deep-research-assistant
2 cd deep-research-assistant
3 npm init -y
```

```
→ ~ mkdir deep-research-assistant
→ ~ cd deep-research-assistant
→ deep-research-assistant npm init -y
Wrote to /Users/guang/deep-research-assistant/package.json:

{
  "name": "deep-research-assistant",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

公众号 · 神光的幸福生活

进入项目，安装依赖：

```
1 pnpm install @langchain/core @langchain/langgraph @langchain/openai
  @langchain/quickjs dedent deepagents dotenv langchain zod
```

写一下 .env 配置：

```
1 OPENAI_API_KEY=sk-xx
2 OPENAI_BASE_URL="https://dashscope.aliyuncs.com/compatible-mode/v1"
3 OPENAI_MODEL="qwen-plus"
4
5 # 用于身份验证，实现链路上报
6 LANGCHAIN_API_KEY=xxx
7 # 指定LangSmith中的项目，追踪结果会归类到该项目下
8 LANGCHAIN_PROJECT=deep-research-assistant
9 # 开启LangSmith追踪功能
10 LANGCHAIN_TRACING_V2=true
11
12 BOCHA_API_KEY=sk-xx
```

开启 langsmith 追踪

这里用到网络搜索，配置下博查的 api key

然后创建 src/tools/search.mjs

```
1 import { tool } from "langchain";
2 import { z } from "zod";
3
4 const BOCHA_API_URL = "https://api.bochaai.com/v1/web-search";
5
6 function formatWebPages(webpages) {
7   return webpages
8     .map(
9       (page, idx) =>
10         `引用: ${idx + 1}
11 标题: ${page.name ?? ""}
12 URL: ${page.url ?? ""}
13 摘要: ${page.summary ?? ""}
14 网站名称: ${page.siteName ?? ""}
15 网站图标: ${page.siteIcon ?? ""}
16 发布时间: ${page.dateLastCrawled ?? ""}` ,
17     )
18     .join("\n\n");
19 }
20
21 async function bochaWebSearch(query, count) {
22   const apiKey = process.env.BOCHA_API_KEY?.trim();
23   if (!apiKey) {
24     return "Bocha 联网搜索的 API Key 未配置（环境变量 BOCHA_API_KEY），请先在 .env
25     中配置后再重试。";
26   }
27   const response = await fetch(BOCHA_API_URL, {
28     method: "POST",
29     headers: {
30       Authorization: `Bearer ${apiKey}`,
31       "Content-Type": "application/json",
32     },
33     body: JSON.stringify({
34       query,
35       freshness: "noLimit",
36       summary: true,
37       count,
38     }),
39   });
40
41   if (!response.ok) {
42     const errorText = await response.text();
43     return `搜索 API 请求失败, 状态码: ${response.status}, 错误信息:
44     ${errorText}`;
45   }
46
47   let json;
48   try {
49     json = await response.json();
50   } catch (e) {
51     return `搜索 API 请求失败, 原因是: 搜索结果解析失败 ${e.message}`;
52   }
53
54   try {
55     if (json.code !== 200 || !json.data) {
56       return `搜索 API 请求失败, 原因是: ${json.msg ?? "未知错误"}`;
57     }
58   }
```

```

56     }
57
58     const webpages = json.data.webPages?.value ?? [];
59     if (!webpages.length) {
60         return `未找到与「${query}」相关的结果。`;
61     }
62
63     return formatWebPages(webpages);
64 } catch (e) {
65     return `搜索 API 请求失败, 原因是: 搜索结果解析失败 ${e.message}`;
66 }
67 }
68
69 export const webSearch = tool(
70   async (input) => {
71     const count = input.count ?? 10;
72     console.log(`🔍 搜索: ${input.query} (${count} 条)`);
73     return bochaWebSearch(input.query, count);
74   },
75   {
76     name: "web_search",
77     description:
78       "使用 Bocha 联网搜索 API 检索互联网网页。输入中文或中英结合的搜索关键词, 可选 count 指定结果数量。返回标题、URL、摘要、网站名称、图标和发布时间。",
79     schema: z.object({
80       query: z
81         .string()
82         .min(1)
83         .describe("搜索关键词, 优先使用中文, 例如: 2026年 AI Agent 框架对比、LangGraph 最新动态"),
84       count: z
85         .number()
86         .int()
87         .min(1)
88         .max(20)
89         .optional()
90         .describe("返回的搜索结果数量, 默认 10 条"),
91     }),
92   },
93 );

```

这个就是网络搜索的 tool

然后写下 4 个 Agent:

src/agent.mjs

```

1  import path from "node:path";
2  import { fileURLToPath } from "node:url";
3  import dedent from "dedent";
4  import { ChatOpenAI } from "@langchain/openai";
5  import { createCodeInterpreterMiddleware } from "@langchain/quickjs";
6  import { createDeepAgent, FilesystemBackend } from "deepagents";
7
8  import { webSearch } from "../tools/search.mjs";
9
10 const projectDir = path.resolve(
11   path.dirname(fileURLToPath(import.meta.url)),

```

```

12  "..",
13  );
14
15  const researcherSubAgent = {
16    name: "researcher",
17    description:
18      "通过联网搜索调研单一子主题。每次只分配一个子主题；多个独立子主题可并行启动多个调研
      员。",
19    systemPrompt: dedent`
20      你是一名专业调研员，负责调研**一个**分配给你的子主题，并写入**一份**调研结果文件。
21
22      ## 工作流程（严格遵守，禁止空转循环）
23
24      1. **可选**：用 write_todos 列出最多 3 条中文执行步骤（例如「搜索官方文档」「搜索
      社区评价」「整理并写入 findings」），然后按步骤执行
25      2. 最多调用 3 次 web_search（硬性上限，绝不超过）
26      3. 将搜索结果整理为结构化摘要，包含关键事实与来源 URL
27      4. 调用 write_file **一次**，保存到任务指定的路径（必须在
      /workspace/sources/findings_*.md 下，禁止写到其他目录）
28      5. 用一句话确认已完成，然后**立即停止**，不要再搜索、写文件或更新 todo
29
30      ## write_todos 使用规则（若使用）
31
32      - 最多 3 条，每条 content 必须用中文
33      - 仅用于拆解本子的调研步骤，不要重复主 Agent 已完成的总体规划
34      - 最后一条 todo 必须是「写入 findings 文件」；该步骤完成后将所有 todo 标为
      completed 并结束
35
36      ## 其他规则
37
38      - 不要重复相同的搜索关键词
39      - write_file 完成后禁止再次搜索——你的任务已结束
40      - 其他人只能看到你写入的文件，内容必须完整、自治
41      - **所有输出必须使用中文**（专有名词如 LangGraph 可保留英文）
42      - 搜索关键词优先使用中文；若主题本身是英文专有名词，可中英结合
43    `,
44    tools: [webSearch],
45  };
46
47  const editorSubAgent = {
48    name: "editor",
49    description:
50      "审阅报告草稿的准确性、结构与完整性。在 /workspace/reports/draft_*.md 写好后使
      用。",
51    systemPrompt: dedent`
52      你是一名资深情报编辑，负责**审阅**报告草稿——**不要**亲自改写报告。
53
54      ## 阅读材料
55
56      - 原始问题：/workspace/sources/question.txt
57      - 待审草稿：任务中指定的路径
58      - 支撑材料：/workspace/sources/ 下的调研文件（如需要）
59
60      ## 审阅要点
61
62      - 报告是否直接回答了原始问题？
63      - 章节结构是否清晰，段落是否充实（而非只有 bullet 列表）？
64      - 是否引用了来源，并在「参考资料」章节列出？

```

```

65     - 是否有遗漏、无依据的断言或缺失的视角？
66     - 语言是否为中文，表述是否专业？
67
68     ## 输出
69
70     返回简洁的审阅意见和具体、可操作的修改建议。
71     **不要**写入报告文件，只提供反馈。所有输出使用中文。
72     `,
73 };
74
75 const analystSubAgent = {
76   name: "analyst",
77   description:
78     "使用 eval REPL 进行数值计算与结构化数据分析。适用于计算、排名、同比对比或
JSON/CSV 分析。",
79   systemPrompt: dedent`
80     你是一名数据分析师，所有计算必须通过 eval REPL 完成——**禁止**猜测数字。
81
82     ## 工作流程
83
84     1. 从 /workspace/sources/ 读取数据文件（或从调研结果中提取数字）
85     2. 在 REPL 中编写并运行 JavaScript，计算总和、均值、排名、增长率等
86     3. 将分析结果保存到 /workspace/sources/analysis_*.md，包含计算逻辑与结论
87
88     必须展示计算过程，结论可从 REPL 输出复现。所有输出使用中文。
89     `,
90   middleware: [createCodeInterpreterMiddleware()],
91 };
92
93 const orchestratorPrompt = dedent`
94   你是「深度调研助手」的主 Agent，负责协调调研、分析与编辑，产出高质量调研简报。
95
96   ## 语言要求
97
98   - **所有输出必须使用中文**：对话回复、write_todos 任务列表、文件内容、搜索关键词
99   - write_todos 中每条 todo 的 content 必须用中文描述，例如「撰写调研计划」「委派调
研员调研 LangGraph」
100   - 搜索时优先使用中文关键词；英文专有名词（如 LangGraph、AutoGen）可保留
101   - 报告、调研笔记、计划文件全部用中文撰写
102
103   ## 你的职责
104
105   协调调研员、分析师和编辑完成报告。不要亲自完成所有调研——将专业工作委派给子 Agent。
106
107   ## 标准流程
108
109   1. **规划** - 用 write_todos 拆解任务（中文）。将用户问题保存到
/workspace/sources/question.txt
110   2. **调研** - 按 web-research 技能：写 research_plan.md，委派调研员子
Agent（可并行）
111   3. **分析** - 若涉及数字对比或数据表，委派分析师子 Agent
112   4. **起草** - **由你亲自**按 report-writer 技能撰写，用 write_file 写入
/workspace/reports/draft_[主题].md
113   5. **审阅** - 委派编辑子 Agent 审稿，根据反馈修订一次
114   6. **定稿** - 保存最终报告到 /workspace/reports/report_[主题]_[日期].md
115
116   ## task 工具（子 Agent 委派）
117

```

```
118  **仅**以下 subagent_type 合法: researcher、analyst、editor、general-
purpose。
119
120  - web-research、report-writer 是**技能**（写作指南），**不是**子 Agent，禁止作
为 subagent_type 调用
121  - 报告起草、修订、定稿由**主 Agent 自己**用 write_file / edit_file 完成，不要委
派 task
122
123  ## 委派规则
124
125  - 每个调研员只负责一个聚焦的子主题
126  - **每份报告最多 3 个调研员**--只选最相关的子主题
127  - 框架对比类任务：优先调研用户明确点名的框架；否则选最重要的 3 个
128  - 最多并行启动 3 个调研员，已有 3 份 findings 文件后不再新增调研员
129  - 仅在确实需要数值计算时使用分析师
130  - 每份报告只调用编辑一次（草稿完成后）
131  - 调研完成后直接进入起草 → 审阅 → 定稿，不要额外开调研轮次
132
133  ## 文件约定
134
135  - 计划与原始资料: /workspace/sources/
136  - 草稿与终稿: /workspace/reports/
137  - 同一时间只编辑一个文件，避免冲突
138
139  ## 完成时告知用户
140
141  - 最终报告保存路径
142  - 2-3 句话的核心发现摘要
143  - 调研中的局限或信息缺口
144 `;
145
146 exportfunction createIntelligenceDeskAgent() {
147   const apiKey = process.env.OPENAI_API_KEY?.trim();
148   if (!apiKey) {
149     thrownewError("未设置 OPENAI_API_KEY 环境变量");
150   }
151
152   const model = process.env.OPENAI_MODEL?.trim() || "gpt-4o";
153   const baseUrl = process.env.OPENAI_BASE_URL?.trim() || undefined;
154
155   const backend = new FileSystemBackend({
156     rootDir: projectDir,
157     virtualMode: true,
158   });
159
160   return createDeepAgent({
161     model: new ChatOpenAI({
162       model,
163       temperature: 0,
164       apiKey,
165       ...(baseUrl
166         ? {
167             configuration: {
168               baseUrl,
169             },
170         }
171       : {}),
172     }),
```



```

173     systemPrompt: orchestratorPrompt,
174     backend,
175     memory: [path.join(projectDir, "AGENTS.md")],
176     skills: ["/skills/"],
177     subagents: [researcherSubAgent, editorSubAgent, analystSubAgent],
178   });
179 }
180
181 export { projectDir };

```

我们直接用 createDeepAgent 的 api，这样不用自己组装 middleware 了，配置下 skills 目录、memory 的文件路径、子 agent 就好了。

这里的 dedent 是去掉换行和缩进的空格，换成 \n 的：

```

systemPrompt: dedent`
  你是一名专业调研员，负责调研**一个**分配给你的子主题，并写入**一份**调研结果文件。

  ## 工作流程（严格遵守，禁止空转循环）

  1. **可选**：用 write_todos 列出最多 3 条中文执行步骤（例如「搜索官方文档」「搜索社区
  2. 最多调用 3 次 web_search（硬性上限，绝不超过）
  3. 将搜索结果整理为结构化摘要，包含关键事实与来源 URL
  4. 调用 write_file **一次**，保存到任务指定的路径（必须在 /workspace/sources/fin
  5. 用一句话确认已完成，然后**立即停止**，不要再搜索、写文件或更新 todo

  ## write_todos 使用规则（若使用）

  - 最多 3 条，每条 content 必须用中文
  - 仅用于拆解本子的调研步骤，不要重复主 Agent 已完成的总体规划
  - 最后一条 todo 必须是「写入 findings 文件」；该步骤完成后将所有 todo 标为 complete

```

写代码的时候正常缩进，用这个可以自动去掉换成 \n

然后分析的子 Agent 需要执行代码，用到了 quickjs 这个 js 引擎来执行：

```

const analystSubAgent = {
  name: "analyst",
  description:
    "使用 eval REPL 进行数值计算与结构化数据分析。适用于计算、排名、同比对比或 JSON/CSV 分析。",
  systemPrompt: dedent`
    你是一名数据分析师，所有计算必须通过 eval REPL 完成——**禁止**猜测数字。

    ## 工作流程

    1. 从 /workspace/sources/ 读取数据文件（或从调研结果中提取数字）
    2. 在 REPL 中编写并运行 JavaScript，计算总和、均值、排名、增长率等
    3. 将分析结果保存到 /workspace/sources/analysis_*.md，包含计算逻辑与结论

    必须展示计算过程，结论可从 REPL 输出复现。所有输出使用中文。
  `,
  middleware: [createCodeInterpreterMiddleware()],
};

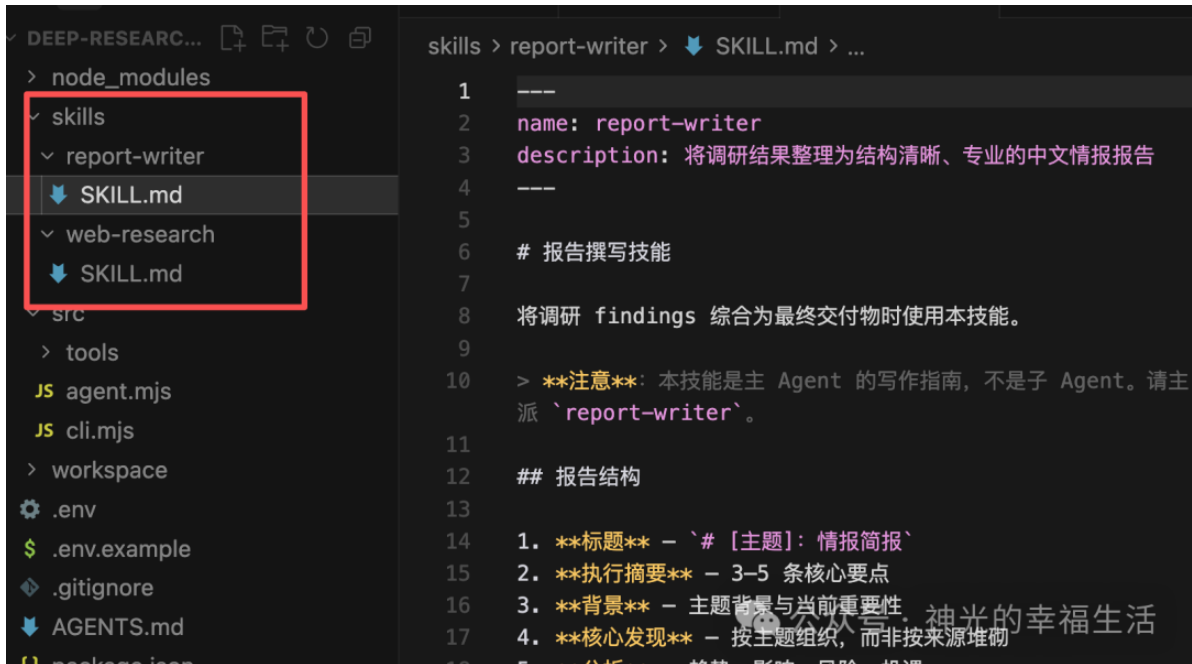
```

公众号 · 神光的幸福生活


```
src > JS agent.mjs > ...
1 import path from "node:path";
2 import { fileURLToPath } from "node:url";
3 import dedent from "dedent";
4 import { ChatOpenAI } from "@langchain/openai";
5 import { createCodeInterpreterMiddleware } from "@langchain/quickjs";
6 import { createDeepAgent, FilesystemBackend } from "deepagents";
7
8 import { webSearch } from "../tools/search.mjs";
```

还有一个 src/cli.mjs 就是调用这个 agent，格式化下输出（直接从仓库复制吧）

还有两个 skill：



```
skills > report-writer > SKILL.md > ...
1 ---
2 name: report-writer
3 description: 将调研结果整理为结构清晰、专业的中文情报报告
4 ---
5
6 # 报告撰写技能
7
8 将调研 findings 综合为最终交付物时使用本技能。
9
10 > **注意**: 本技能是主 Agent 的写作指南，不是子 Agent。请主
11 派 `report-writer`。
12
13 ## 报告结构
14
15 1. **标题** - `# [主题]: 情报简报`
16 2. **执行摘要** - 3-5 条核心要点
17 3. **背景** - 主题背景与当前重要性
18 4. **核心发现** - 按主题组织，而非按来源堆砌
19 5. **分析** - 趋势、影响、风险、机遇
```

skill 就是对 prompt 的封装，这里就是告诉 agent 怎么网络搜索、怎么写报告的

跑一下：

```
1 node src/cli.mjs "调研国家统计局公开的2023年省级地区生产总值（GDP）数据：提取GDP总量前
2 6名省份的具体数值及同比增速，计算六省GDP总和、各省占全国GDP的比重，并按增速从高到低排名"
```

神光的编程秘籍

这样，一个可以加载 skills、有自动读取长期记忆 Agents.md、多个子 Agent 的 Agent 就完成了。

网络搜索的 tool 前面用过几次了。

重点是这个沙箱执行代码的 tool

```
45 });
46 console.log('\n按同比增速排名 (高→低) :');
47 rankedByGrowth.forEach((p, i) => {
48   console.log(`${i+1}. ${p.name}: ${p.growthRate}%`);
49 });
50
51 // 返回结构化数据用于生成表格
52 ({
53   nationalGDP,
54   totalGDP,
55   provinceShares,
56   rankedByGrowth
57 });
```

开始计算分析...

=== 2023年六省GDP数据分析 ===

全国GDP总量: 1260582亿元

大模型不会数学计算，涉及到计算的都是生成代码，用 eval 的 tool 来执行，这里是 js 代码用 quickjs 引擎来执行。

当然，你生成别的语言的代码也行，用对应的引擎执行即可。

这里有个 langsmith 小技巧：

```
45 });
46 console.log('\n按同比增速排名 (高→低) :');
47 rankedByGrowth.forEach((p, i) => {
48   console.log(`${i+1}. ${p.name}: ${p.growthRate}%`);
49 });
50
51 // 返回结构化数据用于生成表格
52 ({
53   nationalGDP,
54   totalGDP,
55   provinceShares,
56   rankedByGrowth
57 });
```

可以通过 filter 过滤出所有的 tool 来，更容易理清流程。

最后就是 todo 了，复杂任务不能走一步想一步，都要提前生成 todo 列表，一步步执行：

主 agent 的 todo 列表:

```
{
  "todos": [
    {
      "content": "保存用户问题到/workspace/sources/question.txt",
      "status": "in_progress"
    },
    {
      "content": "制定调研计划: 确定搜索关键词、数据来源和分析方法",
      "status": "pending"
    },
    {
      "content": "委派调研员子Agent获取国家统计局2023年省级GDP数据",
      "status": "pending"
    },
    {
      "content": "委派分析师子Agent进行数据计算和分析",
      "status": "pending"
    },
    {
      "content": "撰写调研报告草稿",
      "status": "pending"
    },
    {
      "content": "委派编辑子Agent审阅报告",
      "status": "pending"
    }
  ]
}
```

公众号 · 神光的幸福生活

当前的是 in_progress, 完成后会标记为 completed

这个是调研员子 Agent 的 todo 列表:

```
1  {
2    "todos": [
3      {
4        "content": "搜索国家统计局官网发布的《2023年国民经济和社会发展统计公报》",
5        "status": "in_progress"
6      },
7      {
8        "content": "搜索《中国统计年鉴2024》中分省GDP数据",
9        "status": "pending"
10     },
11     {
12       "content": "搜索广东、江苏、山东、浙江、河南、四川六省统计局官网发布的2023年统计公报",
13       "status": "pending"
14     },
15     {
16       "content": "整理GDP总量前6名省份的具体数值、同比增速、数据来源和发布日期",
17       "status": "pending"
18     },
19     {
20       "content": "写入 findings 文件",
21       "status": "pending"
22     }
23   ]
24 }
```

其实这个中间件是 langchain 提供的，我们用一下试试：

src/todo-middleware-test.mjs

```
1 import "dotenv/config";
2 import { ChatOpenAI } from "@langchain/openai";
3 import {
4   createAgent,
5   HumanMessage,
6   todoListMiddleware,
7 } from "langchain";
8
9 const model = new ChatOpenAI({
10 model: process.env.OPENAI_MODEL,
11 apiKey: process.env.OPENAI_API_KEY,
12 temperature: 0,
13 configuration: {
14   baseUrl: process.env.OPENAI_BASE_URL
15 }
16 });
17
18 const agent = createAgent({
19   model,
20   tools: [],
21   systemPrompt:
22     "你是生活规划助手。收到需要多步完成的请求时，先用 write_todos 列出中文执行步骤，然后简要说明你的计划。",
23   middleware: [todoListMiddleware()],
24 });
25
26 const query =
27   "我下周末想带爸妈去杭州玩两天，帮我规划一下：交通怎么选、住哪里方便、必去景点和吃什么，预算控制在人均 1500 元左右。";
28
29 const result = await agent.invoke({
30   messages: [new HumanMessage(query)],
31 });
32
33 console.log("todos:", JSON.stringify(result.todos, null, 2));
34 console.log("-".repeat(50));
35 console.log("回复:", result.messages.at(-1)?.content);
```

用 createAgent + todoListMiddleware

这个中间件自带了 write_todos 的 tool，会生成 todo 列表写到 graph 的 state 里

神光的编程秘籍

这是 langchain 提供的中间件：

```
import {
  createAgent,
  HumanMessage,
  todoListMiddleware,
} from "langchain";
```

公众号 · 神光的幸福生活

我们的 Agent 里也可以用。

通过这个 agent，我们把 deepagents 的 createDeepAgent 的 todo 规划、多 Agent 执行、skill、memory 等用了一遍。

相比自己调用中间件，它集成了各种中间件，内置了对应的 prompt，用起来更简单

ChatOpenAI ID

Feedback Tools **Input** Output Attributes

Input

System

Important To-Do List Usage Notes to Remember

- The `write_todos` tool should never be called multiple times in parallel.
- Don't be afraid to revise the To-Do list as you go. New information may reveal new tasks that need to be added that are irrelevant.

Skills System

You have access to a skills library that provides specialized capabilities and domain knowledge.

Skills Skills: `/skills/` (higher priority)

Available Skills:

- **report-writer:** 将调研结果整理为结构清晰、专业的中文情报报告 → Read `/skills/report-writer/S` instructions
- **web-research:** 结构化多来源联网调研，支持并行委派调研员子 Agent → Read `/skills/web-research` instructions

How to Use Skills (Progressive Disclosure):

Skills follow a **progressive disclosure** pattern - you know they exist (name + description above), but you only get the instructions when needed:

公众号 · 神光的幸福生活

▼ Input

 System**Filesystem Tools** `ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`

You have access to a filesystem which you can interact with using these tools.

All file paths must start with a `/`.

- `ls`: list files in a directory (requires absolute path)
- `read_file`: read a file from the filesystem
- `write_file`: write to a file in the filesystem
- `edit_file`: edit a file in the filesystem
- `glob`: find files matching a pattern (e.g., `**/*.py`)
- `grep`: search for text within files

task (subagent spawner)

You have access to a `task` tool to launch short-lived subagents that handle isolated tasks. These agent they live only for the duration of the task and return a single result.

When to use the task tool:

- When a task is complex and multi-step, and can be fully delegated in isolation

公众号：神光的幸福生活

上下文压缩这块，内置逻辑是每个模型有输入上下文限制，达到 85% 会触发总结，保留 10%

首次调用模型时，按模型 profile 里的 `maxInputTokens` 自动选策略：

场景	触发压缩 (trigger)	压缩后保留 (keep)	工具参数截断
有 <code>maxInputTokens</code>	上下文 85%	保留 10%	同样按 85% / 10%
无 profile (如你项目里的 ChatOpenAI + 兼容 API)	170,000 tokens	最近 6 条消息	超过 20 条消息时截断，保留最近 20 条；单参数最长 2000 字符

公众号：神光的幸福生活

qwen 模型没这个，我们我们可以这样改：

src/max-input-tokens-test.mjs

```
1 import "dotenv/config";
2 import { ChatOpenAI } from "@langchain/openai";
3
4 const model = new ChatOpenAI({
5   model: process.env.OPENAI_MODEL,
6   apiKey: process.env.OPENAI_API_KEY,
7   temperature: 0,
8   configuration: {
```

```

9      baseUrl: process.env.OPENAI_BASE_URL
10    }
11  });
12
13  console.log(model.profile.maxInputTokens);
14
15  Object.defineProperty(model, "profile", {
16    get: () => ({ maxInputTokens: 131_072 }),
17    });
18
19  console.log(model.profile.maxInputTokens);

```

神光的编程秘籍

我们改下这个值，就可以实现对上下文压缩触发阈值的修改：

```

const chatModel = new ChatOpenAI({
  model,
  temperature: 0,
  apiKey,
  ...(baseUrl
    ? {
        baseUrl,
      }
    : {}),
});

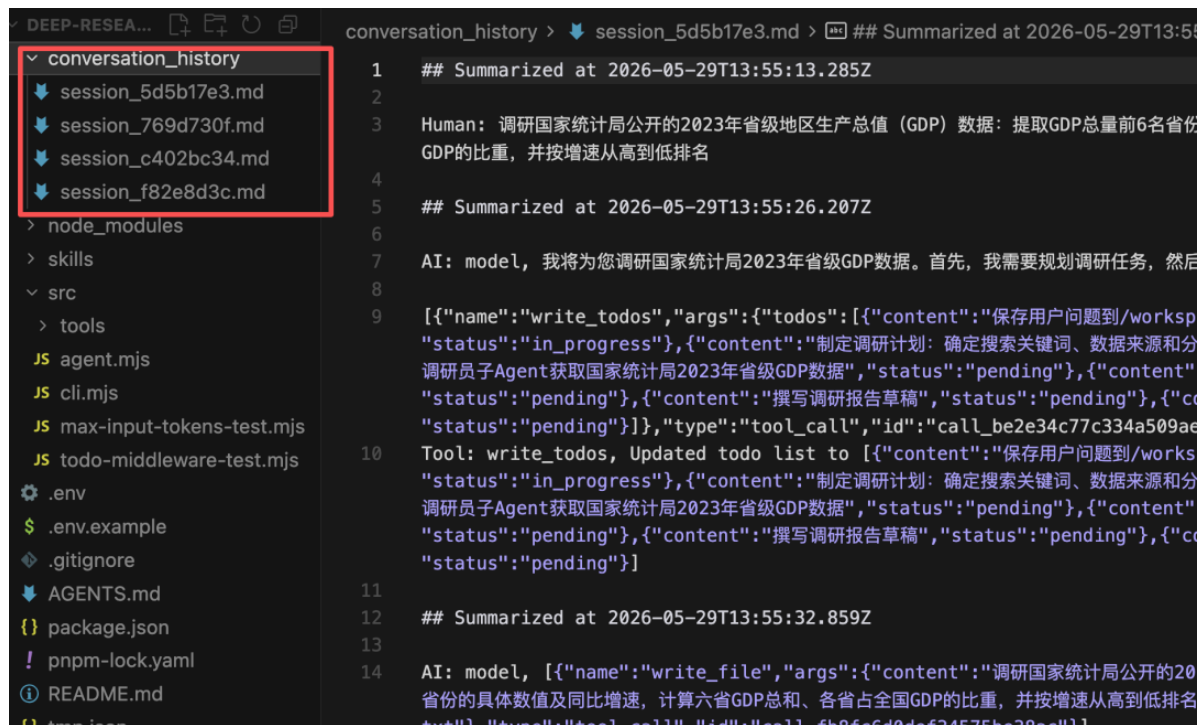
Object.defineProperty(chatModel, "profile", {
  get: () => ({ maxInputTokens: 8_000 }),
});

return createDeepAgent({
  model: chatModel,
  systemPrompt: orchestratorPrompt,
  backend,
});

```



而且，触发摘要后，会把会话原文记录在 conversation_history 目录下归档：



```
DEEP-RESEA... conversation_history > session_5d5b17e3.md > ## Summarized at 2026-05-29T13:55:13.285Z
1 ## Summarized at 2026-05-29T13:55:13.285Z
2
3 Human: 调研国家统计局公开的2023年省级地区生产总值（GDP）数据：提取GDP总量前6名省份
  GDP的比重，并按增速从高到低排名
4
5 ## Summarized at 2026-05-29T13:55:26.207Z
6
7 AI: model, 我将为您调研国家统计局2023年省级GDP数据。首先，我需要规划调研任务，然后
8
9 [{"name": "write_todos", "args": {"todos": [{"content": "保存用户问题到/worksp
  "status": "in_progress"}, {"content": "制定调研计划：确定搜索关键词、数据来源和分
  调研员子Agent获取国家统计局2023年省级GDP数据", "status": "pending"}, {"content":
  "status": "pending"}, {"content": "撰写调研报告草稿", "status": "pending"}, {"co
  "status": "pending"}]}, "type": "tool_call", "id": "call_be2e34c77c334a509ae
10 Tool: write_todos, Updated todo list to [{"content": "保存用户问题到/worksp
  "status": "in_progress"}, {"content": "制定调研计划：确定搜索关键词、数据来源和分
  调研员子Agent获取国家统计局2023年省级GDP数据", "status": "pending"}, {"content":
  "status": "pending"}, {"content": "撰写调研报告草稿", "status": "pending"}, {"co
  "status": "pending"}]}
11
12 ## Summarized at 2026-05-29T13:55:32.859Z
13
14 AI: model, [{"name": "write_file", "args": {"content": "调研国家统计局公开的20
  省份的具体数值及同比增速，计算六省GDP总和、各省占全国GDP的比重，并按增速从高到低排名
  txt"}], "type": "tool_call", "id": "call_fb8fc6d0dcf34575be38ac"}]
```

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

我们基于 DeepAgents 的 createDeepAgent api 实现了深度调研助手。

这是一个多 Agent 架构的 agent

主 Agent 会列出 todo 列表，按步执行，具体的调研、数据计算分析、报告编辑，由三个子 Agent 负责，它们有各自的能力，比如网络搜索、沙盒执行代码

子 Agent 执行的时候，如果需要多个步骤，也是先列 todo 列表再执行，比如调研的时候。执行完更改 todo 任务状态。

Agents.md 的长期记忆、skill 执行 等都是内置了，配置一下就行。

上下文压缩也是内置功能，可以修改 profile.maxInputTokens 来修改触发阈值。

这样，我们没有写很多代码，就完成了了一个多 Agent 架构支持 skill 的功能比较完善的 Agent，这就是 DeepAgents 开发 Agent 的好处，有很多开箱即用的能力。